

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas de Caja Blanca: Análisis de Cobertura y Complejidad Ciclomática				
NÚMERO DE PRÁCTICA:	07	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	15/06/2026	FECHA FIN:	19/06/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: • Una computadora personal • Acceso a Internet con un navegador web • VS Code con lenguaje de programación Python					
DOCENTE(s): • Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: • Aplicar pruebas basadas en la estructura para pruebas de sentencias, decisiones, combinación de condiciones en código fuente • Calcular la complejidad ciclomática de un programa para determinar su complejidad lógica	
TEMAS: • Pruebas de Caja Blanca (basadas en la estructura) • Pruebas de sentencia, ramas, combinación de condiciones • Complejidad Ciclomática	
COMPETENCIAS	C.m. Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

I. Introducción

Las técnicas de prueba de caja blanca (**basadas en la estructura**) consisten en analizar el código y la estructura del producto a probar y utilizar ese conocimiento para realizar las pruebas. Estas técnicas se emplean en la fase de pruebas unitarias, aunque también puede aplicarse en otras etapas, como las pruebas de integración. Para su ejecución, el evaluador o la persona que utilice estas técnicas debe tener un conocimiento profundo de la tecnología empleada en el desarrollo del programa.

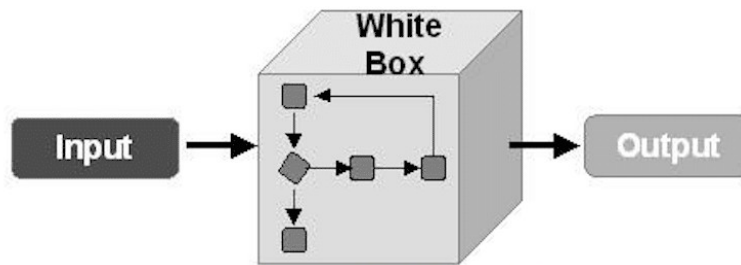


Figura 1 – Técnicas de Caja Blanca. Fuente: instatus

El estándar **ISO/IEC/IEEE 29119-4** es un estándar internacional fundamental para pruebas de software, ya que unifica y estandariza las técnicas de diseño de pruebas. En este estándar se definen técnicas (basadas en la estructura o de "caja blanca") se centran en derivar casos de prueba a partir del código fuente o la estructura interna de un programa.

II. Pruebas de Sentencia (*Statement Testing*)

Busca diseñar casos de prueba de manera que cada instrucción ejecutable del código fuente se ejecute al menos una vez. Es la métrica más débil de cobertura estructural.

Ejemplo de cobertura de sentencias:

		Caso Prueba	Entrada	Salida esperada
1	read(a)	1	7	7
2	IF a > 6 THEN			
3	b = a			
4	ENDIF			
5	print b			

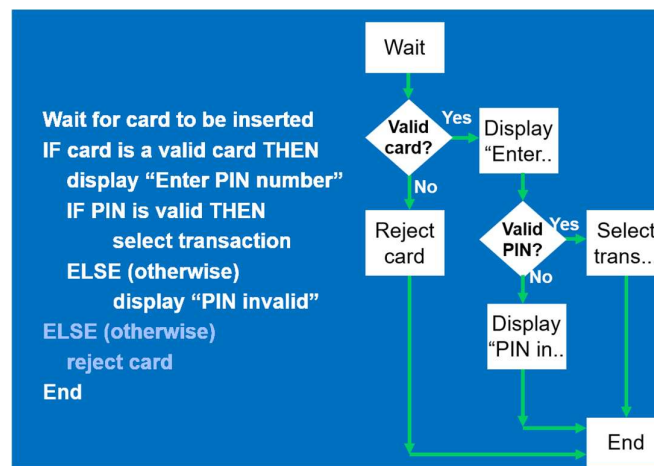
Las 5 sentencias son "cubiertas" por este caso de prueba, se ha logrado 100% de cobertura

Número de sentencias

II. Pruebas de Ramas (*Branch Testing*)

Asegura que cada arista (branch) del Grafo de Flujo de Control (CFG) sea transitada. Esto implica que todas las ramificaciones posibles de las estructuras de control (*if*, *while*, *switch*) se ejecuten.

Ejemplo de cobertura de ramas:



En este ejemplo se necesitan diseñar 3 casos de prueba (ramas) para lograr una cobertura del 100%

III. Pruebas de combinación de condiciones (*Branch condition combination testing*)

También conocida como *Multiple Condition Coverage*. Requiere que se prueben **todas las combinaciones posibles** de Verdadero y Falso de las condiciones atómicas dentro de una decisión. Si hay N condiciones, se requieren $2N$ casos de prueba.

EJERCICIOS RESUELTOS POR EL DOCENTE

Ejercicio 1 – Cálculo de descuento

Dado el código Python: descuento.py

```

def calcular_descuento(edad: int, es_miembro: bool, monto_compra: float) -> float:
    """
    Calcula el precio final aplicando reglas de negocio de descuentos.
    Decisión: (edad >= 65 OR es_miembro) AND (monto_compra >= 1000)
    """
    # Flujo de datos: Definición inicial (Def 1)
    descuento = 0.0

    # Evaluación de la decisión compuesta
    if (edad >= 65 or es_miembro) and (monto_compra >= 1000):
        # Flujo de datos: Redefinición si la regla se cumple (Def 2)
  
```

```
    descuento = 0.15
else:
    # Flujo de datos: Redefinición alternativa (Def 3)
    descuento = 0.00

# Flujo de datos: Uso de la variable 'descuento' (Uso)
precio_final = monto_compra - (monto_compra * descuento)

return precio_final
```

Diseñar las pruebas unitarias con pytest para lograr una cobertura del 100% aplicando las técnicas de pruebas de sentencia, ramas y combinación de condiciones.

```
import pytest
from descuento import calcular_descuento

# =====
# 1 & 2. STATEMENT TESTING y BRANCH TESTING
# =====
# Para lograr un 100% en ambas métricas, solo necesitamos
# ejecutar cada línea de código (Statement) y transitar
# por las aristas True y False del 'if' (Branch).
# Lo logramos con solo 2 casos de prueba.

def test_statement_branch_camino_verdadero():
    """Ejecuta el bloque del IF (15% de descuento)"""
    assert calcular_descuento(
        edad=70, es_miembro=False, monto_compra=1500) == 1275.0

def test_statement_branch_camino_falso():
    """Ejecuta el bloque del ELSE (Sin descuento)"""
    assert calcular_descuento(
        edad=30, es_miembro=False, monto_compra=500) == 500.0

# =====
# 3. BRANCH CONDITIONS COMBINATION TESTING
# =====
# Decisión: (C1 OR C2) AND C3
# C1: edad >= 65
# C2: es_miembro == True
# C3: monto_compra >= 1000
# Requisito: Probar las 8 (2^3) combinaciones de V y F.
```

```
@pytest.mark.parametrize(
    "edad, miembro, monto, esperado, tabla_verdad",
    [
        # 1. C1=V, C2=V, C3=V -> Resultado: True (Aplica Descuento)
        (70, True, 1500, 1275.0, "V-V-V"),

        # 2. C1=V, C2=V, C3=F -> Resultado: False (No Aplica)
        (70, True, 500, 500.0, "V-V-F"),

        # 3. C1=V, C2=F, C3=V -> Resultado: True (Aplica Descuento)
        (70, False, 1500, 1275.0, "V-F-V"),

        # 4. C1=V, C2=F, C3=F -> Resultado: False (No Aplica)
        (70, False, 500, 500.0, "V-F-F"),

        # 5. C1=F, C2=V, C3=V -> Resultado: True (Aplica Descuento)
        (30, True, 1500, 1275.0, "F-V-V"),

        # 6. C1=F, C2=V, C3=F -> Resultado: False (No Aplica)
        (30, True, 500, 500.0, "F-V-F"),

        # 7. C1=F, C2=F, C3=V -> Resultado: False (No Aplica)
        (30, False, 1500, 1500.0, "F-F-V"),

        # 8. C1=F, C2=F, C3=F -> Resultado: False (No Aplica)
        (30, False, 500, 500.0, "F-F-F"),
    ]
)

def test_branch_conditions_combination(edad, miembro, monto, esperado, tabla_verdad):
    """
    Evalúa la matriz completa exhaustiva de las condiciones lógicas.
    """
    assert calcular_descuento(edad, miembro, monto) == esperado
```

Ejecute el siguiente comando en el terminal:

```
pytest test_descuento.py -v --cov=descuento --cov-branch
```

Verifique sus resultados:

```

plugins: cov-7.1.0
collected 10 items

test_descuento.py::test_statement_branch_camino_verdadero PASSED [ 10%]
test_descuento.py::test_statement_branch_camino_falso PASSED [ 20%]
test_descuento.py::test_branch_conditions_combination[70-True-1500-1275.0-V-V-V] PASSED [ 30%]
test_descuento.py::test_branch_conditions_combination[70-True-500-500.0-V-V-F] PASSED [ 40%]
test_descuento.py::test_branch_conditions_combination[70-False-1500-1275.0-V-F-V] PASSED [ 50%]
test_descuento.py::test_branch_conditions_combination[70-False-500-500.0-V-F-F] PASSED [ 60%]
test_descuento.py::test_branch_conditions_combination[30-True-1500-1275.0-F-V-V] PASSED [ 70%]
test_descuento.py::test_branch_conditions_combination[30-True-500-500.0-F-V-F] PASSED [ 80%]
test_descuento.py::test_branch_conditions_combination[30-False-1500-1500.0-F-F-V] PASSED [ 90%]
test_descuento.py::test_branch_conditions_combination[30-False-500-500.0-F-F-F] PASSED [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.1-final-0 -----
Name           Stmts   Miss Branch BrPart  Cover
-----
descuento.py      8      0      2      0   100%
TOTAL             8      0      2      0   100%
===== 10 passed in 0.26s =====

```

IV. Complejidad Ciclomática

La **Complejidad Ciclomática** es una métrica de ingeniería de software (desarrollada por Thomas McCabe en 1976) que mide la cantidad de **caminos linealmente independientes** a través del código fuente de un programa.

A nivel académico, no mide el total de caminos que existen, sino la cantidad de "caminos base". Cualquier otro camino posible en el código se puede formar combinando estos caminos base. Sirve directamente para definir el **límite superior del número de casos de prueba** necesarios para lograr una cobertura completa de ramas (Branch Coverage).

Se calcula mediante la fórmula del grafo de flujo de control:

$$CC = A - N + 2$$

$$CC = P + 1$$

Donde: A = aristas, N = nodos de grafo de flujo, P = número de nodos predicado (nodos que contienen una condición)

Complejidad Ciclomática	Evaluación del Riesgo
1 - 10	Programa simple, sin mucho riesgo
11 - 20	Más complejo, riesgo moderado
21 - 50	Complejo, programa de alto riesgo
> 50	Programa no testeable, muy alto riesgo

Como desarrollador enfocado en la calidad del código y las pruebas, medir la **complejidad ciclomática** es fundamental. Mantener este número bajo no solo hace que tu código sea más legible, sino que reduce drásticamente la cantidad de pruebas unitarias que necesitas escribir para cubrir todos los caminos posibles.

En el ecosistema de Python, usaremos la herramienta Radon para medir la complejidad lógica de un programa.

Radon categoriza la complejidad ciclomática en rangos que van de la **A** (código simple, bajo riesgo) a la **F** (código inmanejable, requiere refactorización inmediata).

- **Instalación:** pip install radon
- **Uso básico en terminal:** radon cc mi_codigo.py -s (el flag -s te muestra el score numérico exacto).

EJERCICIOS RESUELTOS POR EL DOCENTE

Ejercicio 2 – código de bajo riesgo (CC<10)

```
def simple(a, b):  
    if a > 0:  
        if b == 0:  
            print("sin valor")  
        else:  
            print(b)  
            if a > 21:  
                print(a)
```

Calcular la complejidad ciclomática:

```
radon cc codigo_simple.py -s -a
```

```
codigo_simple.py  
F 1:0 simple - A (4)  
  
1 blocks (classes, functions, methods) analyzed.  
Average complexity: A (4.0)
```

Ejercicio 2 – código de alto riesgo (CC>10)

```
def validar_sistema_seguridad(password, usuario, intentos, ip_bloqueada, admin):  
    score = 0  
  
    # 1. Primer punto de decisión (if)  
    if ip_bloqueada:  
        return "Acceso denegado: IP Bloqueada"  
  
    # 2. Segundo punto de decisión (if)  
    if intentos > 3:  
        return "Cuenta temporalmente suspendida"  
  
    # 3. Tercer punto de decisión (if)  
    if len(password) < 8:  
        return "Contraseña demasiado corta"
```

```
# 4-7. Cuatro bucles para analizar caracteres (+4 CC)
for car in password:      # Corregido: 'in' en lugar de 'en'
    if car.isupper():      # 5. Punto de decisión (+1 CC)
        score += 2
    if car.isdigit():      # 6. Punto de decisión (+1 CC)
        score += 2
    if car in "@#$%&*":    # 7. Punto de decisión (+1 CC)
        score += 3

# 8. Octavo punto de decisión (if)
if usuario == "admin":
    # 9. Noveno punto de decisión (if anidado, +1 CC)
    if not admin:
        return "Fraude: Nombre admin sin privilegios"
    score += 5

# 10. Décimo punto de decisión (if)
if score < 10:
    return "Contraseña insegura"
# 11. Undécimo punto de decisión (elif)
elif score < 15:
    return "Seguridad Media"
else:
    return "Seguridad Alta"
```

Calcular la complejidad ciclomática:

```
radon cc codigo_complejo.py -s -a
```

```
codigo_complejo.py
F 1:0 validar_sistema_seguridad - C (12)

1 blocks (classes, functions, methods) analyzed.
Average complexity: C (12.0)
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 9

III. EJERCICIOS/PROBLEMAS PROPUESTOS

EJERCICIOS PROPUESTOS:

Ejercicio 1 – Diseño de pruebas de sentencias, rama y combinación de condiciones

Elija un programa libre en Python (entre 100 a 500 LOC) y diseñe las pruebas unitarias basadas en enfoque de caja blanca para lograr una cobertura del 100%

Ejercicio 2 - Guerra de Testers – Parte III

Con su equipo de testers de caja blanca debe identificar 03 funciones de software bajo prueba para hallar la complejidad ciclomática. Elabore el grafo de flujo por cada función, halle la CC y luego compare su resultado con la herramienta Radon si el programa está escrito en Python o su equivalente en otro lenguaje de programación.

I. CUESTIONARIO

Pregunta de discusión 01:

Ustedes son el equipo líder de pruebas en una empresa de desarrollo. El Gerente General, que no es técnico, acaba de leer un artículo sobre métricas de software y emite una nueva directiva inamovible: *"A partir de mañana, ningún sistema pasa a producción si no tiene 100% de cobertura de ramas (Branch Coverage). Si llegamos al 100%, garantizamos cero bugs y despedimos a la mitad del equipo de QA para ahorrar costos"*.

Pregunta de discusión: ¿Cómo prepararían una defensa técnica y financiera para demostrarle al Gerente que está cayendo en una peligrosa ilusión? Construyan y expliquen un ejemplo de código en Python o Java muy sencillo que alcance el 100% de cobertura de ramas, pero que aun así falle catastróficamente al ejecutarse en producción.

Pistas para su investigación:

- Revisen el concepto de "Functional testedness vs. Structural testedness".
- ¿Qué tipos de errores (ej. concurrencia, memoria, requisitos ausentes) no detecta la cobertura de código?

Pregunta de discusión 02:

Existe un debate en calidad de software. Algunos equipos afirman: *"Si probamos exhaustivamente todas las entradas y salidas del sistema como usuarios finales (caja negra) y todo funciona perfecto, examinar el código fuente (caja blanca) es una pérdida de tiempo comercial"*.

Pregunta de discusión: Utilicen la analogía de un automóvil o de una cirugía médica para debatir esta afirmación. Busquen e investiguen al menos un caso histórico de desastre de software donde un análisis de caja blanca y estructura de código podría haber prevenido una tragedia que la caja negra pasó por alto.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 10

Pistas para su investigación:

- Investiguen conceptos como "código muerto" (*dead code*) y puertas traseras lógicas (*logic backdoors*).
- Discutan cómo caja blanca encuentra errores de seguridad (vulnerabilidades) que la funcionalidad pura no revela.

Pregunta de discusión 03:

Son el equipo de ingeniería de aseguramiento de la calidad de producto. Al escanear el código heredado, descubren que la función central procesar_ticket_cliente() tiene una Complejidad Ciclomática de 88, ya que maneja asignaciones, prioridades, notificaciones y escalamientos en un solo bloque gigante de código.

Pregunta de discusión: Escribir 88 casos de prueba unitaria para una sola función es un antipatrón insostenible. Como ingenieros de software, ¿cómo utilizarían el concepto de "Refactorización" y los principios SOLID para reducir esta complejidad? Si dividen esta función gigante en 10 funciones más pequeñas, ¿la Complejidad Ciclomática total del sistema desaparece, se divide, o simplemente se traslada a otro tipo de pruebas (como las de integración)? Argumenten los pros y contras de esta decisión.

Pistas para su investigación:

- Investiguen el Principio de Responsabilidad Única (SRP) y cómo impacta directamente en las métricas de prueba.
- Analicen el "Teorema de la Conservación de la Complejidad" en la arquitectura de software.
- Discutan cómo una alta complejidad ciclomática afecta la mantenibilidad cuando un cliente pide un cambio rápido en la lógica de negocio.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

[1] Spillner, A., *Software Testing Foundations*, 5th Ed., 2021

[2] Myers, G., *The Art of Software Testing*, 3rd Ed., 2012

[3] ISO/IEC/IEEE 29119-4, *Técnicas de Pruebas de Software*, 2021

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y proporcionan explicaciones claras, incluyen citas.	La mayoría de las respuestas son correctas, pero algunas carecen de la profundidad esperada, pocas citas.	Algunas respuestas son correctas, pero incompletas o con falta de claridad en las explicaciones, citas no relevantes.	La mayoría de las respuestas son escuetas y/o incompletas, no hay evidencia de haber buscado información que las respalden, sin citas.	No presenta el cuestionario.
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.



UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA



Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación

Aprobación: 2022/03/01

Código: GUIA-PRLD-001

Página: 12